

הרחבת Tracepath

לתמיכה ב IOAM

Enhancing Tracepath IOAM support

מבצעים:

Ariel Itzkovich
Omer Reuven

אריאל איצקוביץ
עומר ראובן

מנחה:

Tal mizrahi

טל מזרחי

סמסטר רישום: חורף תשפ"ד

תאריך הגשה: יוני, 2025

תוכן עניינים

3		רקע וסקירת ספרות
8		בניית האמולטור
9		הטמעת IOAM בTracepath
11		יצירת מעטפת לtracepath
12		תוצאות:
13		הסבר הקוד של scapy parser:
16		סיכום ועבודה להמשך

רקע וסקירת ספרות

מהו ioam?

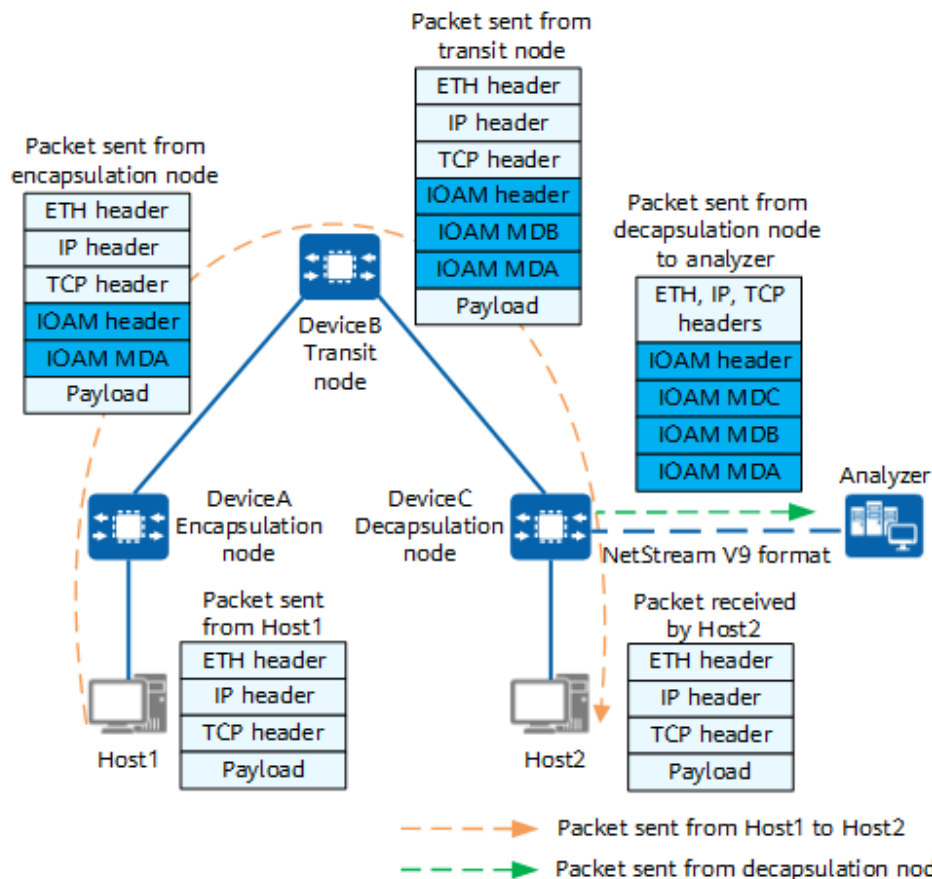
IOAM – In-situ Operations, Administration, and Maintenance הוא תקן המאפשר ניטור של רשת תקשורת, וקבלת מידע אודותיה כגון עומס בתחנות מסוימות, השהיות, המסלול בו עוברת החבילה ועוד. בניגוד לשיטות אחרות בהן על מנת לנטר את הרשת מועברות חבילות נוספות, מה שעלול לשנות את אופי התקשורת, ב-IOAM המידע מוטמע בתוך החבילה כ-`metadata` נוסף במקום ייעודי בחבילה.

איך הוא עובד?

IOAM פועל בתת-רשת (דומיין) אשר מוגדרת על ידי מתחזק הרשת לעבוד עם הפרוטוקול. רק אם כל התחנות בדומיין מוגדרות לתמוך הוא יעבוד כהלכה, ויסיפו לה את המידע הדרוש, כפי שהוגדר לה. תחנות שלא מוגדרות לעבוד עם הפרוטוקול עלולות להשמיט את ה-`header`, מה שעלול לגרום לאובדן מידע לאורך המסלול.

כל תחנה שתומכת בפרוטוקול תוסיף לחבילות העוברות דרכה מידע על עצמה, בהתאם למה שהוגדר מראש על ידי מתחזק הרשת, בהתאם לשדות שהפרוטוקול מאפשר ולתמיכה של הרשת – לדוגמה: מזהה התחנה (Node ID), מזהי ממשקים (Ingress/Egress), חותמת זמן, עומק תור, או כל שדה אחר שתואם ל-`trace-type` שהוגדר.

כך, בכל גישה לחבילה – גם אם היא לא הגיעה ליעד – ניתן לפענח את כלל המידע שהתווסף לאורך המסלול, ולהבין באיזה דרך עברה החבילה ומה היה מצב הרשת בכל שלב.



ישנם מספר דרכי פעולה אפשריות בו יכול לעבוד הפרוטוקול, כאשר כל דרך מכתיבה איך המידע יתווסף לחבילה:

- **Pre-allocated Trace** - החבילה יוצאת עם מקום קבוע מראש לכל התחנות לאורך המסלול. כל תחנה כותבת למקום בזכרון שהוקצה בחבילה מראש. **זהו הסוג בו השתמשנו בפרויקט**, מכיוון שהוא שומר על גודל קבוע של החבילה ומונע חריגות MTU.
- **Incremental Trace** - כל תחנה מוסיפה את המידע שלה בצורה דינמית, כך שה-extension header וגדל עם כל קפיצה.
- **Edge-to-Edge** - איסוף מידע רק בתחנות הקצה של המסלול, לרוב למטרות תצפית כוללת על הרשת.
- **Proof of Transit** - תחנות חותמות על כך שהחבילה עברה דרכן, לצורך הבטחת מסלול.
- **Direct Export** - המידע לא נכתב לתוך החבילה אלא נשלח החוצה ממנה שיטות אלו מאפשרות גמישות למתחזק הרשת הרוצה לנתח מצבים שונים.

מבנה IOAM

IOAM מועבר על גבי פרוטוקול ipv6 באמצעות שני סוגי extension headers ש-ipv6 מאפשר: destination option header ו hop by hop extension header.

מבנה החבילה:

תחילה ישנו ipv6 header כאשר במקרה שלנו שדה הnext header יכול את הערך המתאים לסוג הheader הבא, כלומר Hop-by-Hop, או Destination Option.

-ipv6 header

Version	Traffic class	Flow label	
Payload length		Next header	Hop limit
Source address			
Destination address			

Extension header משורשרים באופן הבא:

IPv6 header	TCP header +Data
next header TCP	

IPv6 header	Hop By Hop Header	TCP header +Data
next header Hop By Hop	next header TCP	

עבור תמיכה מלאה בIOAM יש לתמוך בפענוח של שני סוגי הheader ובפרויקט שלנו נשתמש רק בסוג אחד: hop by hop שכן בחרנו לעבוד עם pre-allocated בלבד. לכן עלינו יהיה למצוא את סוג הheader הרלוונטי לברר את סוגו ולחלץ ממנו את המידע.

מבנה extension header

הextension header מגיע מיד לאחר הipv6 header ומכיל את השדות הרלוונטים לפי סוגו מאלה הקיימים בפרוטוקול, מכיוון שאנו עובדים עם hop by hop extension header נתמקד בו.

שני השדות הראשונים הם חלק מהמבנה של IPv6.

Next Header - מתאר את סוג הheader הבא.

Hdr Ext Len - אורך הheader.

מיד לאחר מכן מגיעים שדות הoption שיכולים להכיל יותר מoption אחד לכן על מנת לפענח IOAM יש לזהות את האפשרות המתאימה.

השדות הבאים הם: (שדות הקיימים בכלל האופציות בפרוטוקול)

Option type - סוג האופציה במקרה שלנו IOAM preallocated- יכיל את הערך 0x31

Option length - אורך האופציה.

ב IOAM option ישנו שדה פנימי נוסף:

IOAM type - סוג הפעולה של ioam במקרה שלנו pre-allocated

לפני שדה זה יהיו מספר ביטים "שמורים" לשדות עתידיים

next header	header len	option type	option len
reserved	IOAM type	option data	

כעת הפקטה הנשלחת מתחילה לעבור בIOAM domain שהוגדר ומתמלאת במידע בכל מעבר בתחנה כותבת.

IOAM Data

לאופציית הioam יש כמה אפשרויות המזוהות לפי שדה IOAM Type

Pre-allocated Trace Option

גודל מוגדר של זכרון מוקצה אליו נכתב המידע מהתחנות מוקצה מראש בחבילה כל תחנה כותבת אל אזור מסויים עד למילוי.

Incremental Trace Option

כל תחנה מוסיפה את המידע שלה לאחר בלוק המידע האחרון שנכתב כשהזכרון מוקצה דינמית, והאופציה הולכת וגדלה.

Proof of Transit Option

תחנות מוסיפות חתימה או מזהה שמאשר כי החבילה עברה בהן.

-Edge-to-Edge Option

המידע נכתב רק בתחנות הקצה של הדומיין ולא בתחנות ביניים.

המבנה של המידע ב header מורכב משדות המשתנים בהתאם לסוג הפעולה שנבחר נתמקד בסוג בו אנו משתמשים preallocated בו השדות פנימיים העיקריים באופציות הם:

Namespace- מזהה הדומיין של ioam ברשת

NodeLen- אורך המידע שתכתוב כל תחנה, גודל זה תלוי כמובן במידע שנבחר שיכתבו התחנות.

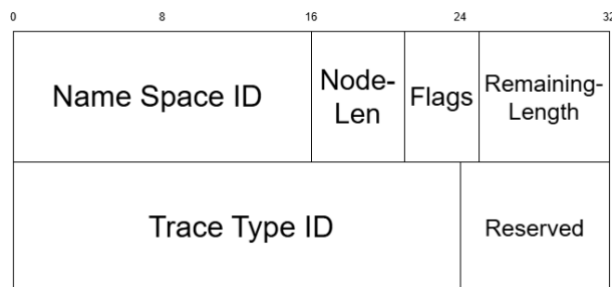
Flags – מסיכת דגלים בגודל byte במקרה שלנו רק אחד מהם פעיל.

RemainingLen – כמות בלוקים פנויים לכתיבה על ידי תחנות הביניים.

IOAM Trace-Type - 24 ביטים המזהים את שדות המידע שיישמרו בכל כתיבה על ידי תחנות הביניים, כאשר כל ביט מגדיר שדה שונה והפרוטוקול מגדיר את אלו שדות אפשריים ובאיזה סדר.

Reserved - ביטים שמורים מאופסים על ידי encapsulating noden.

לאחר מכן מגיע המידע עצמו, רשימת בלוקי המידע שנרשמים או יירשמו על ידי תחנות הביניים בהתאם למה שהוגדר ב ioam trace type סדר הכתיבה הוא מהסוף להתחלה כך ניתן לנתח את התחנות לאורך המסלול כולו.



רקע על tracepath

מהו Tracepath

Tracepath היא פונקציית לינוקס המשמשת לניתוח מסלול החבילות מהתחנה כלשהיא אל תחנת יעד אחרת, מטרתה מציאת סדר תחנות הביניים בין תחנת המוצא לתחנת היעד יחד עם זמני הrtt אל כל תחנה בדרך.

איך עובדת פונקציית tracepath?

Tracepath עובדת על ידי שליחת חבילת udp icmp עם שדה hop limit אשר מתחיל מערך 1 ועולה ב 1 עד אשר מתקבלת תשובה מתחנת היעד. המנגנון עובד כך שכאשר תחנת ביניים מקבלת את ההודעה עם hoplimit=1 היא מפילה אותה ומחזירה icmp time exceeded אל תחנת המוצא. בגלל אופן העלאה של שדה hoplimit בכל פעם נגלה שכבה חדשה ברשת עד לתחנת היעד כך ניתן למפות את המסלול.

הכלי עובד באמצעות שימוש ב datagram socket כלי פשוט יחסית שמעביר הודעות ללא בקרה ואיננו מעביר את כל extension header בשלמותם, זה כמובן מספיק לשימוש אותו tracepath תוכנן לעשות אך לא להרחבה אותה תכננו לבצע.

```

[root@server ~]# tracepath www.google.com
1?: [LOCALHOST]
1: gateway
1: gateway
2: 192.168.0.1
3: 192.168.1.1
4: 192.168.1.1
4: 172.31.34.18
5: no reply
6: 172.31.210.142
7: 172.31.210.98
8: 10.10.10.10
9: ws82-230-252-122.rcil.gov.in
10: ws81-230-252-122.rcil.gov.in
11: 172.31.210.113

```

	pmtu 1500
	0.285ms
	0.635ms
	2.054ms asymm 64
	2.293ms asymm 63
	2.336ms pmtu 1492
	11.719ms asymm 62
	18.869ms asymm 60
	15.146ms asymm 59
	12.970ms asymm 58
	11.040ms asymm 57
	23.002ms asymm 56
	31.713ms asymm 55

בניית האמולטור

לצורך הרצת הכלי שבנינו יצרנו אמולציה של רשת פשוטה באמצעות linux network namespaces

כל namespace מתנהג כמו מחשב נפרד עם משאבים נפרדים וכך ניתן לחבר אותם אחד לשני

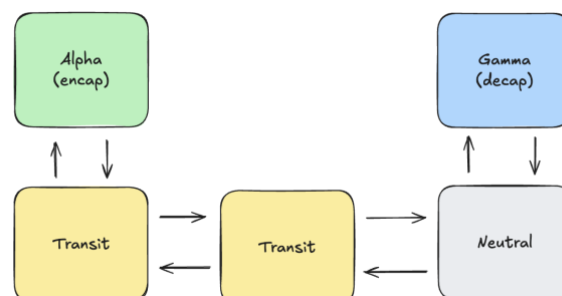
בצורה וירטואלית על מנת לדמות רשת.

יצרנו אמולציה הכוללת שתי תחנות קצה ושלושה ראטרים כל אחת מהתחנות והראטרים הוא

namespace בפני עצמו והחיבורים ביניהם נעשים באמצעות ממשק veth חיבור ethernet וירטואלי.

הגדרנו את כולם לעבוד עם ioam וכולם נמצאים באותו ioam domain כך שיש להם גישה לאותו

ioam headern הראטור השלישי באמולציה רק מעביר מידע ולא כותב מידע משלו.



באמולציה שלנו:



הטמעת IOAM בTracepath

ioam הוא פרוטוקול חדש באופן יחסי ומתפתח ועדיין לא מוטמע בכלים נפוצים, לכן בחרנו לעבוד עם tracepath שהוא כלי נפוץ ורובסטי, וגם בעל מבנה פשוט יחסית לכלים אחרים ולכן הנחנו שיהיה קל לעבוד איתו.

Tracepath מציג מידע בסיסי בלבד על טופולוגיית הרשת ועל זמן הRTT שלקח להגיע לכל תחנה רשת התומכת בIOAM תוסיף את המידע אל החבילה אך אין דרך קלה להציג אותו הרעיון להרחיב את tracepath כך שתפענח ותציג את המידע שהוטמע לאורך הדרך כחלק אינטגרלי מהכלי.

מנגנון התמיכה ב-IOAM

על מנת לאפשר קריאה של המידע המוטמע בחבילות נצטרך לחלץ אותו מתוך החבילות החוזרות.

נבצע את הצעדים הבאים

- מתוך הpayload של הודעת icmp החוזרות נזהה האם נכלל hop by hop extension header.
- נזהה את סוג האופציה מתוך שדה option type ונאמת כי זוהי אופציית IOAM-0x31.
- מתוך האופציה נחלץ את השדות של header:
 - Namespace – זיהוי הדומיין
 - Trace type זיהוי השדות שנשמרו על ידי תחנות הביניים וכתובות בבלוקים
 - Node Len – אורך כל בלוק התואם לגודל השדות שנשמרו
- נעבור על כל הבלוקים המלאים ומכל אחד מהם נחלץ את השדות המופיעים בtracetype וזאת על ידי חישוב המיקום של כל אחד מהם בבלוק על ידי גודל השדות.

שינוי tracepath – גישה ישירה

תחילה בחרנו לשנות ישירות את הכלי tracepath שכן קיים בו כבר מנגנון בסיסי לטיפול בהודעות חזרות ובנוסף, הוא קוד פשוט יחסית וקל לעבוד מולו. לאחר מכן זיהינו כי הכלי משתמש בsocket מסוג sock_dgrm אשר מתווך על ידי שכבת ICMP של הקרנל ולא מחזיר את החלק של payload שכולל את המידע שאנו צריכים אלה רק בתים ספורים.

הוספת SOCK_RAW

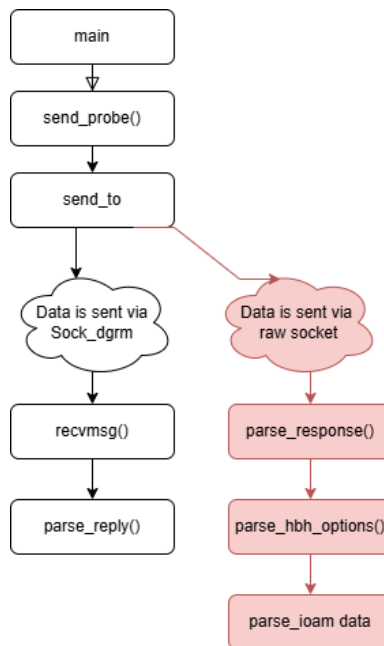
על מנת לעקוף את מגבלת הגישה, נבחנה האפשרות להוסיף מנגנון נוסף חליפי שיקלוט את החבילות החוזרות כאשר אלו מצופות להכיל מידע ioam לשם כך השתמשנו בsocket מסוג SOCK_Raw המאפשר קליטת כל תוכן החבילה כולל header בשלמותם. גישה זו אכן אפשרה גישה לhop by hop header ולמידע ioam שהוטמע בו אך דרש התערבות משמעותית במימוש הקיים לtracepath ולמעשה יצר מנגנון שלם המקביל למנגנון הקיים. החלק המשמעותי לאחר קבלת החבילה הוא חילוץ המידע ממנה שחייב להעשות באופן דינמי בגלל שמבנה המידע שנשמר תלוי בשדות שנבחרו בtrace type:

כל ביט שמופעל מייצג שדה אחר שיש לכתוב ולקרוא אותו

Bit #	Field Name	Size (Bytes)
0	Hop_Lim+Node_ID	4
1	Ingress_IF + Egress_IF	4
2	Timestamp Seconds	4
3	Timestamp Fraction (Ns)	4
4	Transit Delay	4
5	Namespace-Specific Data	4
6	Queue Depth	2
7	Checksum Complement	2
8	Hop_Lim + Node_ID (wide)	8
9	Ingress_IF + Egress_IF (wide)	8
10	Namespace-Specific Data (wide)	8
11	Buffer Occupancy	4
12-21	Reserved	4
22	Opaque State Snapshot	Variable
23	Reserved	—

parsern מבצע קריאה של השדה tracetype ובהתאם לביטים הדולקים בו מחשב את אורך הבלוק ומפענח ממנו את השדות בהתאם.

תרשים של המבנה הקיים של tracepath והתוספת עבור IOAM:



תוצאת ביניים

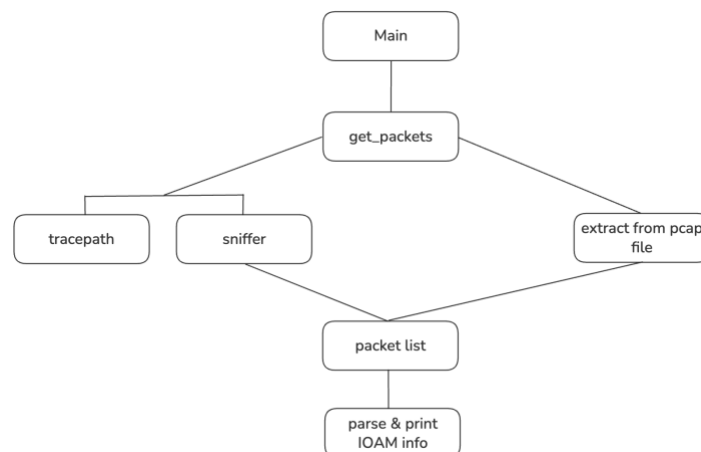
המעבר ל raw socket אפשר גישה לשדות אך הצריך שינוי נרחב של המימוש הקיים הוספת מנגנון מקביל לפענוח ההודעות וparser לניתוחן כל זה בקוד שלא יועד לכך מראש. בשלב זה הופיעו בעיות יציבות והאטה זמני rtts ולכן חשבנו כי עדיף יהיה להפריד בין המימוש של tracepath לבין הכלי שיבצע את parsing כך שכל חלק יעבוד בצורה מיטבית עם כלים המתאימים לו

Tracepath- בצורה הקיימת והרובסטית

Parser- ימומש בPython- scapy באופן שיאפשר תחזוקה קלה וגמישות.

יצירת מעטפת לtracepath

בשלב זה כתבנו כלי שיפעיל את tracepath חיצונית בשביל לקבל חבילות icmp חוזרות עם המידע הדרוש או יקרא חבילות מתוך pcap נתון שכבר אמור להכיל את החבילות עם מידע ioam.



בדרך זו אנו משתמשים בtracepath כפי שהוא ועוטפים אותו בצורה קלה לתחזוקה וגמישה לשינויים עתידיים שנרצה להוסיף. כמו כן לצורך פענוח החבילות השתמשנו בפונקציות קיימות של python scapy שמתאימות לפענוח חבילות מה שהפך את הקוד ליציב יותר ואת העבודה והתחזוקה לנוחה יותר.

ניתן להשתמש בפונקציה על ידי הפעלתה על קבצי pcap קודמים או על ידי הרצת tracepath על הרשת.

1. Parse from .pcap file:

```
sudo python3 ioam_parser.py alpha.pcap
```

2. Live capture on a network interface (for IPv6 destination):

```
sudo python3 ioam_parser.py -i <interface_name> <IPv6_address>
```

כמו כן ישנה פונקציה המייצרת את סימולציה ולאחריה מריצה את הפונקציה ומנתחת את התוצאות כך המשתמש יכול לוודא כי הסביבה שלו מתאימה להרצת הפונקציה.

```
sudo ./netns_emu.sh
```

תוצאות:

התוצאות המפוענחות נשמרות לקובץ ioam_data.txt

```
ICMPv6DestUnreach from db04::2
Namespace ID: 123
Node Length: 24
Trace Type: 0xbc1000
Node Array:
Node 1:
Hop_Lim and Node ID (short):
Hop_Lim: 4
ID: 0x000001
Timestamp Seconds: 2106-02-07 06:28:15
Timestamp Fraction: 1.000000000s
Transit Delay: 0xffffffff
Namespace Data (short): 0xdeadbee0
Buffer Occupancy: 0xffffffff
Node 2:
Hop_Lim and Node ID (short):
Hop_Lim: 3
ID: 0x000002
Timestamp Seconds: 2025-10-03 18:50:26
Timestamp Fraction: 0.000069024s
Transit Delay: 0xffffffff
Namespace Data (short): 0xdeadbee1
Buffer Occupancy: 0xffffffff
Node 3:
Hop_Lim and Node ID (short):
Hop_Lim: 2
ID: 0x000004
Timestamp Seconds: 2025-10-03 18:50:26
Timestamp Fraction: 0.000069025s
Transit Delay: 0xffffffff
Namespace Data (short): 0xdeadbee2
Buffer Occupancy: 0xffffffff
```

בדוגמה זו ניתן לראות מידע מהתחנה הסופית

ניתן לקרוא ממנה את המידע על כל אחת מתחנות הביניים לפי השדות שנבחרו בתחילת הריצה
.tracetype
ומידע בסיסי על התחנה עצמה.
באמצעות כלי זה אפשר לקבל מידע נוסף בקלות על תחנות הביניים וזרימת הפקטות דרכן.

הסבר הקוד של scapy parser:

מבט על:

```
292 def main():
293     args = parse_args()
294     packets = get_packets(args)
295     ioam_info_array = parse_packets(packets)
296     print_ioam_info(ioam_info_array)
```

פרסור האגומנטים נועד להבין אם לקרוא מpcap file או להריץ tracepath, get_packets מבצע זאת. Parse_packets מוציא את המידע ומכניס אותו לרשימה ואז מדפיסים אותה.

הרצת tracepath:

```
253 def capture_tracepath_packets(args):
254     ready = threading.Event()
255     sniffer = AsyncSniffer(
256         filter="icmp6",
257         iface=args.interface,
258         timeout=30,
259         store=True,
260         started_callback=lambda: ready.set(),
261     )
262     logger.info(f"Starting Scapy Sniffer")
263     sniffer.start()
264     ready.wait()
265
266     run_tracepath(args.input)
267
268     time.sleep(0.1)
269     sniffer.stop()
270     sniffer.join()
271     packets = sniffer.results
272     logger.info(f"Scapy Sniffer Closed, {len(packets)} Packets Found")
273     return packets
```

לפני ההרצה של tracepath פותחים סניפר אסינכרוני בעזרת scapy כדי לקלות חבילות חוזרות.

ההרצה עצמה בעזרת subprocess:

```
239 def run_tracepath(destination):
240     logger.info("Running Tracepath...")
241     try:
242         result = subprocess.run(
243             ["tracepath", "-6", "-n", "-m 20", "-l 256", "-p 33434", destination],
244             capture_output=True,
245             text=True,
246             check=True,
247         )
248         logger.info("Tracepath Output:\n\n" + add_indent(result.stdout, 1))
249     except subprocess.CalledProcessError as e:
250         logger.error(f"Tracepath Error:\n\n {add_indent(e.stderr, 1)}")
```

הגבלנו את גודל החבילות ל-256 עם הדגל -l בעיקר כי מצאנו בעיה עם tracepath ו-ioctl שבא כשtracepath מנסה לחשב mtu והוא לא יודע שיוסיפו לו headers נוספים חיצוניים שהוא לא מכיר אז הוא לא מצליח לחשב את mtu כמו שצריך וזה מכניס אותו ל-inf loop של ניסיון מחדש לשלוח חבילות (שנפלות כי הן גדולות מידי)

זיהוי חבילות ICMP עם ioam hbh header בתוך ipv6 header בתוך icmp payload:

```
200 def parse_packet(pkt) -> IOAMPacketInfo | None:
201     if ICMPv6TimeExceeded in pkt:
202         icmp_payload = bytes(pkt[ICMPv6TimeExceeded].payload)
203         icmp_str = "ICMPv6TimeExceeded"
204     elif ICMPv6DestUnreach in pkt:
205         icmp_payload = bytes(pkt[ICMPv6DestUnreach].payload)
206         icmp_str = "ICMPv6DestUnreach"
207     else:
208         return None
209
210     try:
211         inner_ipv6 = IPv6(icmp_payload)
212         if IPv6ExtHdrHopByHop in inner_ipv6:
213             hopopts = inner_ipv6[IPv6ExtHdrHopByHop]
214             for opt in hopopts.options:
215                 if opt.otype == 0x31: # IOAM Trace Option
216                     logger.info(
217                         f"IOAM Option Detected in {icmp_str} Packet (Length: {len(opt)})"
218                     )
219                     ioam_header = parse_ioam_option(opt)
220                     if ioam_header:
221                         return IOAMPacketInfo(icmp_str, pkt[IPv6].src, ioam_header)
222     except Exception as e:
223         logger.error(f"Failed to extract IOAM: {e}")
```

זו בעצם הסיבה הכי גדולה לשימוש ב-scapy על פני פרסור ידני עם כתובות ב-a שקל לטעות בו. חבילות ICMPv6TimeExceeded הן חבילות חוזרות לחבילות שנגמר להן ttl בדרך ICMPv6DestUnreach אלה חבילות תגובה לאלה שהגיעו ליעד.

בשביל לפרסר את הoption עצמו השתמשנו במבנה מהrfc וגם בwireshark ופרסרנו כל byte לפי המשמעות שלו:

```
175 def parse_ioam_option(opt) -> IOAMHeader | None:
176     data = opt.optdata
177
178     option_type = data[1]
179     if option_type != 0:
180         logger.warning("Non Pre-Allocated Trace-Option Detected, Skipping")
181         return
182
183     namespace_id = int.from_bytes(data[2:4])
184     # node length is first 5 bits of 5th byte, scale of 4
185     node_length = 4 * ((data[4] & 0b1111000) >> 3)
186     # remaining length is 7 bottom bits of 6th byte, scale of 4
187     remaining_length = 4 * (data[5] & 0b0111111)
188
189     trace_type = data[6:9]
190     trace_fields = decode_trace_type(trace_type)
191
192     # skip free space
193     trace_data = data[10:]
194     node_array = trace_data[remaining_length:]
195
196     header = IOAMHeader(namespace_id, node_length, trace_type, node_array, trace_fields)
197     return header
```

מבנה הנתונים שמייצג את הביטים של tracetype, שמגדיר את סוג השדות בכל node:
(הערכים נלקחו מתוך הrfc: <https://datatracker.ietf.org/doc/rfc9197> בסעיף 4.4.1)

```
32 class TraceFieldEnum(Enum):
33     HOP_LIM_NODE_ID_SHORT = (0, "Hop_Lim and Node ID (short)", 4)
34     INGRESS_EGRESS_IF_ID_SHORT = (1, "Ingress and Egress IDs (short)", 4)
35     TIMESTAMP_SECS = (2, "Timestamp Seconds", 4)
36     TIMESTAMP_FRAC = (3, "Timestamp Fraction", 4)
37     TRANSIT_DELAY = (4, "Transit Delay", 4)
38     NAMESPACE_SPECIFIC_SHORT = (5, "Namespace Data (short)", 4)
39     QUEUE_DEPTH = (6, "Queue Depth", 4)
40     CHECKSUM_COMPLEMENT = (7, "Checksum Complement", 4)
41     HOP_LIM_NODE_ID_WIDE = (8, "Hop_Lim and Node ID (wide)", 8)
42     INGRESS_EGRESS_IF_ID_WIDE = (9, "Ingress and Egress IDs (wide)", 8)
43     NAMESPACE_SPECIFIC_WIDE = (10, "Namespace Data (wide)", 8)
44     BUFFER_OCCUPANCY = (11, "Buffer Occupancy", 4)
45
46     def __init__(self, bitnum: int, field_name: str, size: int):
47         self.bitnum = bitnum
48         self.field_name = field_name
49         self.size = size
```

ואז בעת קבלת trace_type פשוט מוסיפים את struct המתאים מתוך enum לפי הביטים הדלוקים:

```
164 def decode_trace_type(trace_type) -> list[TraceFieldEnum]:
165     trace_fields = []
166     trace_type_int = int.from_bytes(trace_type)
167
168     for tf in TraceFieldEnum:
169         if trace_type_int & (1 << (23 - tf.bitnum)): # MSB is bit 0
170             trace_fields.append(copy.copy(tf))
171
172     return trace_fields
```

סיכום ועבודה להמשך:

יצרנו פונקציה המאפשרת למשתמש הרצה נוחה, פענוח והצגה של מידע ioam המתווסף לחבילה במהלך הרצת פונקציית tracepath. בחנו גישה של שינוי פנימי בפונקציה וגישה של עטיפה חיצונית של פונקציית tracepath לבסוף בחרנו בגישה העטיפה החיצונית. המימוש כעטיפה חיצונית מאפשר למשתמש גישה נוחה לשדות ויכולת להוסיף יכולות כמו ויזואליזציות של המידע שדות נוספים אוטומציות נוספות ועוד וזאת לעומת שינוי פנימי שהיה מגביל את המימוש ומקשה את ההתממשקות עם הפתרון שלנו.

ביבליוגרפיה

- Brockners, F., Bhandari, S., & Mizrahi, T. (2022). *Data Fields for In Situ Operations, Administration, and Maintenance (IOAM)*. RFC 9197. IETF.
<https://datatracker.ietf.org/doc/rfc9197/>
- Deering, S., & Hinden, R. (2017). *Internet Protocol, Version 6 (IPv6) Specification*. RFC 8200
<https://datatracker.ietf.org/doc/html/rfc8200>
- Huawei – What is IOAM?
<https://info.support.huawei.com/info-finder/encyclopedia/en/IOAM.html>
- Data Fields for In Situ Operations, Administration, and Maintenance (IOAM) RFC 9197:
<https://datatracker.ietf.org/doc/rfc9197/>

